

Programação no R - Aula 8

Disciplina: Lógica da programação (de computadores) e
análise de dados no R

Prof. Maurício Garcia de Camargo. IO-FURG.

2025-10-31



Dataframes (DF) no R

Você certamente já sabe:

- O que é um dataframe?

Linhas e colunas de qualquer tipo de dado (número ou texto)

- Criar uma dataframe usando função `data.frame`

```
data.frame(vet1, vet2, vet3)
```



Criar colunas em um DF

```
1 DF = data.frame(col1=0:5,col2=10:15)
2 DF
```

	col1	col2
1	0	10
2	1	11
3	2	12
4	3	13
5	4	14
6	5	15

```
1 DF$nova_col = LETTERS[1:6] # O vetor de letras LETTERS vem embutido no R
2 DF
```

	col1	col2	nova_col
1	0	10	A
2	1	11	B
3	2	12	C
4	3	13	D
5	4	14	E
6	5	15	F



Filtrar linhas e colunas do DF

Extrair elementos do DF (linha,coluna):

```
1 DF
```

	col1	col2	nova_col
1	0	10	A
2	1	11	B
3	2	12	C
4	3	13	D
5	4	14	E
6	5	15	F

```
1 DF[3,2] # linha 2, coluna 1
```

```
[1] 12
```



Dataframes (DF) no R

- Extrair colunas do DF:

```
1 DF
```

```
  col1 col2 nova_col  
1     0   10        A  
2     1   11        B  
3     2   12        C  
4     3   13        D  
5     4   14        E  
6     5   15        F
```

```
1 DF$col1
```

```
[1] 0 1 2 3 4 5
```

```
1 # ou
```

```
2 DF[,1]
```

```
[1] 0 1 2 3 4 5
```



Dataframes (DF) no R

- Extrair linhas do DF:

```
1 DF
```

	col1	col2	nova_col
1	0	10	A
2	1	11	B
3	2	12	C
4	3	13	D
5	4	14	E
6	5	15	F

```
1 DF[1,]
```

	col1	col2	nova_col
1	0	10	A



Filtros de linhas e colunas de um DF

```
1 DF
```

	col1	col2	nova_col
1	0	10	A
2	1	11	B
3	2	12	C
4	3	13	D
5	4	14	E
6	5	15	F

```
1 DF[DF$col1>3 , c(1,2)]
```

	col1	col2
5	4	14
6	5	15

```
1 DF[DF$col1 > 2 & DF$col2 > 14 ,]
```

	col1	col2	nova_col
6	5	15	F



Criando um fator no DF

Variáveis categóricas são chamadas de **fatores** (*factors*) no R, e as categorias são chamadas de **níveis** (*levels*).

```
1 fat = c(rep('Pré-adol',3) , rep('Adol',3))
2 fat
```

```
[1] "Pré-adol" "Pré-adol" "Pré-adol" "Adol"      "Adol"      "Adol"
```

```
1 DF$fator = fat
2 DF
```

	col1	col2	nova_col	fator
1	0	10	A	Pré-adol
2	1	11	B	Pré-adol
3	2	12	C	Pré-adol
4	3	13	D	Adol
5	4	14	E	Adol
6	5	15	F	Adol



Funções para criar e testar fatores

```
1 is.factor(DF$fator) #False
```

```
[1] FALSE
```

```
1 DF$fator = as.factor(DF$fator) #Transforma o vetor num fator  
2 is.factor(DF$fator) #True
```

```
[1] TRUE
```

```
1 DF$fator
```

```
[1] Pré-adol Pré-adol Pré-adol Adol      Adol      Adol  
Levels: Adol Pré-adol
```



Funções específicas para DF

```
1 head(DF, 3)          # 2 primeiras linhas
```

	col1	col2	nova_col	fator
1	0	10	A	Pré-adol
2	1	11	B	Pré-adol
3	2	12	C	Pré-adol

```
1 tail(DF, 2)          # 3 últimas
```

	col1	col2	nova_col	fator
5	4	14	E	Adol
6	5	15	F	Adol

```
1 dim(DF)
```

```
[1] 6 4
```

```
1 nrow(DF)
```

```
[1] 6
```

```
1 ncol(DF)
```

```
[1] 4
```



Funções específicas para DF

```
1 names(DF)           # nomes das colunas
```

```
[1] "col1"      "col2"      "nova_col" "fator"
```

```
1 str(DF)             # estrutura (tipos por coluna)
```

```
'data.frame':  6 obs. of  4 variables:
 $ col1      : int  0 1 2 3 4 5
 $ col2      : int 10 11 12 13 14 15
 $ nova_col  : chr  "A" "B" "C" "D" ...
 $ fator     : Factor w/ 2 levels "Adol","Pré-adol": 2 2 2 1 1 1
```

```
1 summary(DF)         # resumo estatístico
```

col1	col2	nova_col	fator
Min. :0.00	Min. :10.00	Length:6	Adol :3
1st Qu.:1.25	1st Qu.:11.25	Class :character	Pré-adol:3
Median :2.50	Median :12.50	Mode :character	
Mean :2.50	Mean :12.50		
3rd Qu.:3.75	3rd Qu.:13.75		
Max. :5.00	Max. :15.00		



Exercício 1

Para o exercício 1, você vai usar o pacote `readxl`, que não vem com o R base.

O pacote pode ser usado de 2 maneiras, após acertar o `WorkDir`:

Assim:

```
library(readxl)  
plan = read_excel('plan1.xlsx')
```

Ou...

```
plan = readxl::readexcel('plan1.xlsx')
```



Voltando ao exercício 1

1.1. Baixe a planilha 1 (plan1.xlsx) na página da disciplina.

1.2. Armazene esta planilha num objeto tipo `dataframe` chamado `dados` usando o pacote `readxl`.

Lembre-se de acertar o WorkDir usando o menu *Session>Set Working Directory*.

1.3. Crie 3 novos dataframes a partir de `dados`:

- fat (local e estacao)
- abio (sal, temp e pH)
- bio(sp1, sp2 e sp3)



Voltando ao exercício 1

- 1.4. Crie uma nova coluna no dataframe `dados` chamada `abund` correspondente à abundância total das espécies em cada amostra.
- 1.5. Crie uma nova coluna chamada `riqueza` contendo a riqueza de espécies, ou seja, o número de espécies em cada amostra.
- 1.6. Crie um novo dataframe chamado `indices` contendo os novos índices de abundância e riqueza recém criados.



Voltando ao exercício 1

1.7. Extraia as médias de salinidades para cada **local**.

Sugestão: crie um filtro para cada local e extraia as médias individualmente.

1.8. Extraia as médias de temperatura para cada estação.

TEM QUE HAVER FORMA MAIS AMIGÁVEL DE FAZER ISSO!



Apresentando a função `aggregate()`

`aggregate()` resume uma ou mais colunas por **grupos**.

Exemplos de perguntas que ela resolve:

- média da temperatura por local
- desvio padrão da temperatura por local
- média da salinidade por local **E** estação;
- soma da abundância por estação;
- número de observações por local.



Sintaxe básica da função **aggregate()**

Existem várias maneiras de montar os argumentos da função.

A forma mais poderosa é usando as fórmulas do R:

```
1 # aggregate(variavel ~ grupos,  
2 #           data = dados,  
3 #           FUN = funcao)
```

onde:

variavel: o que será resumido (ex.: Temp)

grupos: por quem será agrupar (ex.: Local, ou Local + Estacao)

data: o dataframe que será resumido **FUN:** função de resumo (ex.: mean, sum, length, sd)



Exemplos com a nossa planilha **dados**

- 1. Média da temperatura por local

```
1 # aggregate(temp ~ local, data = dados, FUN = mean)
```

o R junta todas as linhas de cada local e calcula a média.

- 2. Média da salinidade por local **E** estação

```
1 # aggregate(sal ~ local + estacao, data = dados, FUN = mean)
```

Agora cada linha da saída é um grupo (Local × Estacao).



Exemplos com a nossa planilha **dados**

- 3. Soma da abundância por estação

```
1 # aggregate(spl ~ estacao, data = dados, FUN = sum)
```

- 4. Quantas amostras tenho por local?

```
1 # aggregate(temp ~ local, data = dados, FUN = length)
```

Aqui a função **length** apenas conta quantas linhas existem por local.



Resumo para várias variáveis ao mesmo tempo

Quando tem várias variáveis para resumir juntas (Temp, Sal, pH), se usa outro formato da `aggregate()`:

```
1 # aggregate(dados[, c("temp", "sal", "pH")],  
2 #           by = list(Local = dados$local,  
3 #                     Estacao = dados$estacao),  
4 #           FUN = mean)
```

Onde:

dados[, c(...)] → quais colunas serão resumidas;

by = list(...) → por quem serão agrupadas;

FUN = mean → função do resumo.



Exercício 2

2.1 Abra a planilha `limno.xlsx` na página da disciplina.

2.2. Armazene esta planilha num objeto tipo `dataframe` chamado `dad` usando o pacote `readxl`.

2.3. Crie 3 novos dataframes a partir de `dad`:

- fat (ponto, estacao e setor)
- abio (temp, pH, etc)
- bio(Bosmina, Bosminopsis, Calanoida, etc)

2.4. Crie uma coluna para a abundância total por amostra.

2.5. Separe uma planilha para cada estação do ano.

2.6. Extraia a média da combinação de `estacao` e `setor`.



